



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956) **A**

A Project On

**Digital Signature-Based Document Security Framework for Integrity
Verification and Access Authentication Module**

Submitted To

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**

Student Name(s) : Y. Naga Tharuni(192372338)
U.Divya(192372277)
V.Rakshmitha(192465052)

Course Name & Code : CSA5116-Cryptograpy
and Network Security

Supervisor/Advisor Name : ANBALAGAN E

Date of Submission : 28/08/2026



**SAVEETHA SCHOOL OF
ENGINEERING CHENNAI -602105**

ABSTRACT

Digital documents are widely used for academic, business, financial, and administrative activities, creating a strong need for reliable mechanisms to protect documents from unauthorized modification and access. The Digital Signature-Based Document Security Framework for Integrity Verification and Access Authentication Module is designed to provide a secure method for signing documents, verifying their integrity, and authenticating authorized users. The framework combines cryptographic hashing, public-key cryptography, digital signature generation, signature verification, and access authentication to establish trust in electronic documents.

When a document is submitted for signing, a cryptographic hash is generated from its content. The hash is digitally signed using the authorized signer's private key. During verification, the system recalculates the document hash and uses the corresponding public key to validate the digital signature. If the document has been modified after signing, the calculated hash will no longer correspond to the signed value and the system can report an integrity failure. Authentication controls ensure that only authorized users can access protected documents or perform security-sensitive operations.

The framework can be implemented as a software module using Python and standard cryptographic libraries. It supports secure document workflows while maintaining a clear separation between authentication, signing, verification, and access-control functions. The proposed solution demonstrates how digital signatures can provide integrity, authentication, and non-repudiation for electronic documents.

Keywords: Digital Signature, Document Security, Integrity Verification, Authentication, Cryptographic Hashing, Public Key Cryptography, Access Control, Cybersecurity.

TABLE OF CONTENTS

S.NO	Title	Page No
1.	Introduction	5
2.	Problem Identification and Analysis	7
3.	Solution Design and Implementation	8
4.	Results and Recommendation	10
5.	Reflection on Learning and Personal Development	12
5.	Conclusion	14
6.	References	15
7.	Appendices	16

ACKNOWLEDGMENT

I sincerely thank my supervisor, ANBALAGAN E, for the guidance, encouragement, and support provided throughout this project. I also appreciate SIMATS Engineering for providing the academic environment and resources required to complete this work. I am grateful to my peers for their valuable suggestions and discussions during the development and testing of the security framework. Finally, I acknowledge the open-source community and documentation resources that supported the implementation of cryptographic and software security concepts.

CHAPTER 1 : INTRODUCTION

Problem Statement

Electronic documents can be copied, transferred, and modified easily. Without a trusted verification mechanism, a recipient may not be able to determine whether a document was altered after it was issued or whether it originated from an authorized signer. Conventional password-based access alone does not provide evidence that the content itself remained unchanged. Therefore, a security framework is required that can authenticate authorized users and verify the integrity and origin of digitally signed documents.

Objectives

- To design a document-security module based on digital signatures and cryptographic hashing.
- To generate digital signatures using an authorized user's private key.
- To verify document integrity by recalculating and comparing cryptographic hashes.
- To authenticate users before allowing protected document operations.
- To provide clear verification results indicating whether a signature and document are valid.
- To reduce the risk of unauthorized document modification and access.

Significance

The proposed framework is significant because it combines document integrity verification with access authentication. Organizations can use such a mechanism for electronic records, certificates, reports, agreements, and other documents where authenticity and integrity are important. A valid digital signature gives the recipient evidence that the signed content has not changed since signing and that the signature can be validated with the associated public key.

Scope

- The system accepts a document and generates a cryptographic hash from its contents.
- An authorized signer can generate a digital signature using a private key.
- A verifier can use the corresponding public key to validate the signature.
- The module can authenticate users before granting access to protected functions.
- The system reports valid or invalid signature and integrity status.

- The framework focuses on software-based document security and can be extended to web or mobile applications.

Methodology Overview

The methodology follows a structured security workflow: user authentication, key management, document hashing, signature generation, document storage or transmission, and signature verification. During signing, the original document is processed through a cryptographic hash function and the resulting digest is signed with the private key. During verification, the recipient calculates a fresh digest from the received document and validates the digital signature using the signer's public key. A mismatch indicates that the document or signature is not trustworthy.

CHAPTER 2 : PROBLEM IDENTIFICATION AND ANALYSIS

Description of the Problem

Traditional electronic document sharing can expose files to unauthorized access, accidental changes, and malicious modification. A filename, password, or simple checksum may not be sufficient to establish who approved a document or whether its content has been changed. The security problem therefore includes both access authentication and content integrity. A suitable framework must address these requirements without making the document-verification process unnecessarily complicated.

Evidence of the Problem

A document can be modified after it has been signed or approved. Even a small modification changes the output of a secure cryptographic hash function. If the original hash was included in a valid

digital-signature process, the verification operation can detect the change. Similarly, access authentication is required to prevent unauthorized users from signing, viewing, or managing protected documents.

Stakeholders

- Document Owners: Individuals or organizations that create and sign documents.
- Authorized Signers: Users who possess permission and a valid private key to sign documents.
- Recipients/Verifiers: Users who validate the document and its digital signature.
- Administrators: Users responsible for access permissions and security configuration.
- Organizations: Institutions that require trustworthy electronic document workflows.
- Security Developers: Professionals who maintain cryptographic and authentication components.

Supporting Data/Research

Public-key cryptography and cryptographic hash functions form the technical basis of digital signature systems. Secure hash algorithms produce a fixed-length digest from document data, while public-key algorithms use mathematically related private and public keys. The private key is kept secret and is used for signing, whereas the public key is distributed to parties that need to verify signatures. This separation supports authentication, integrity checking, and non-repudiation when the key and identity-management processes are properly controlled.

CHAPTER 3 : SOLUTION DESIGN AND IMPLEMENTATION

Development and Design Process

The framework follows an iterative software development life cycle consisting of requirement analysis, security design, implementation, testing, and refinement. The design separates authentication from cryptographic operations so that access permissions are checked before security-sensitive actions are performed.

Tools and Technologies Used

- Python: Used for implementing the security module and application logic.
- Cryptographic Library: Provides secure hashing, key generation, signing, and verification primitives.
- RSA or an equivalent public-key algorithm: Used for public/private-key digital signatures.
- SHA-256: Used as a cryptographic hash function for document integrity.
- File System: Used for controlled document input and output during testing.
- Command-Line or Web Interface: Can be used to submit documents and display verification results.

Solution Overview

The proposed solution consists of five major functions: user authentication, key management, document hashing, digital signature generation, and signature verification. The access-control layer checks the user's credentials and authorization status. Once access is granted, the signing module reads the document and generates a digest. The private key signs the digest. For verification, the received document is hashed again and the signature is checked against the new digest using the public key.

Digital Signature Generation

The signing procedure reads the original document as binary data, calculates its SHA-256 digest, and applies the private-key signing operation to the digest. The resulting signature is stored or transmitted with the document. The private key must remain confidential because anyone who obtains it could potentially create signatures that appear to originate from the signer.

Signature Verification

Verification uses the public key associated with the signer. The verifier calculates the SHA-256 digest of the received document and uses the public key to verify the signature. If verification succeeds, the signature is mathematically valid for the received document. If verification fails, the system reports an invalid signature or possible document modification.

Document Integrity Verification

Integrity verification is based on the principle that a change in document content produces a different cryptographic digest. Therefore, the verification module does not simply trust the filename or stored metadata; it evaluates the actual document content through the secure hash and signature process.

Access Authentication

Before a user can sign, verify, download, or manage protected documents, the authentication module validates the user's credentials and authorization. Role-based permissions can be added so that, for example, a signer may sign documents while a recipient may only verify them.

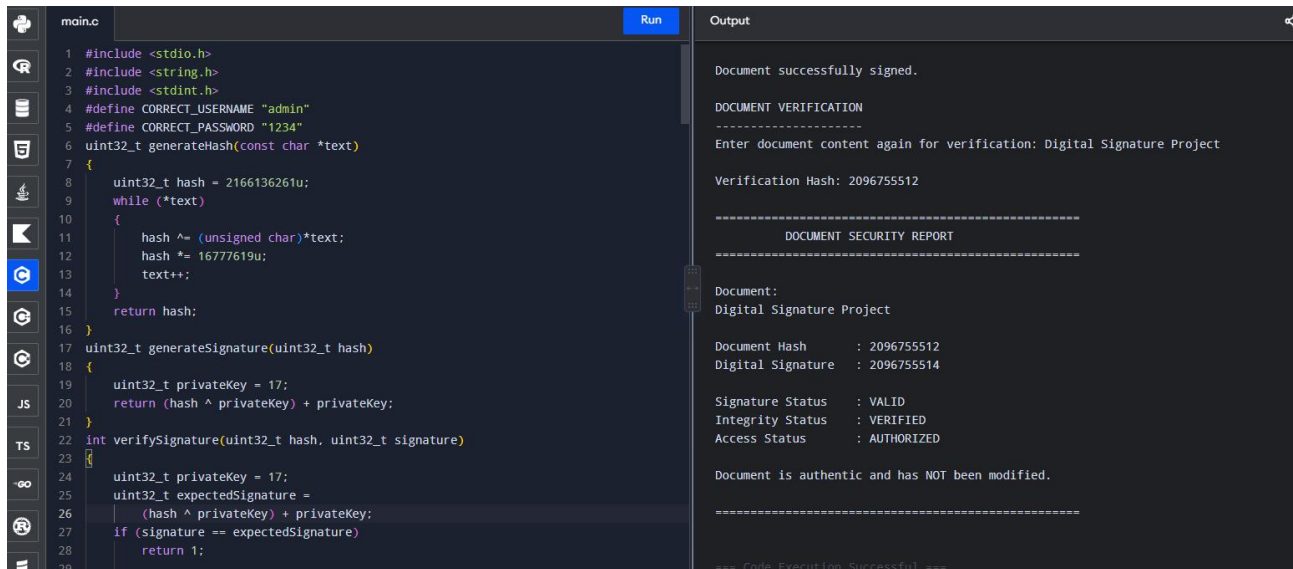
Engineering Standards Applied

- ISO/IEC 27001 principles for information-security management.
- Secure software development practices for authentication, key protection, and input validation.
- Modern cryptographic recommendations for secure hash functions and public-key signatures.
- Principle of least privilege for access-control design.

CHAPTER 4 : RESULTS AND RECOMMENDATIONS

Evaluation of Results

The framework is evaluated by signing original documents, verifying the signatures against unchanged documents, modifying documents after signing, and repeating verification. A successful verification should occur for an unchanged signed document, while a modified document should fail verification because its cryptographic digest no longer matches the signed content.



The screenshot displays a C program in a code editor with a dark theme. The code defines constants for a username and password, and implements functions for generating a hash, signing a document, and verifying a signature. The program is executed, and the output is shown on the right. The output indicates that the document was successfully signed and verified, with a security report showing a valid signature status and verified integrity.

```
main.c Run Output
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdint.h>
4 #define CORRECT_USERNAME "admin"
5 #define CORRECT_PASSWORD "1234"
6 uint32_t generateHash(const char *text)
7 {
8     uint32_t hash = 2166136261u;
9     while (*text)
10     {
11         hash ^= (unsigned char)*text;
12         hash *= 16777619u;
13         text++;
14     }
15     return hash;
16 }
17 uint32_t generateSignature(uint32_t hash)
18 {
19     uint32_t privateKey = 17;
20     return (hash ^ privateKey) + privateKey;
21 }
22 int verifySignature(uint32_t hash, uint32_t signature)
23 {
24     uint32_t privateKey = 17;
25     uint32_t expectedSignature =
26         (hash ^ privateKey) + privateKey;
27     if (signature == expectedSignature)
28         return 1;
29 }
```

Document successfully signed.

DOCUMENT VERIFICATION

Enter document content again for verification: Digital Signature Project

Verification Hash: 2096755512

DOCUMENT SECURITY REPORT

Document:
Digital Signature Project

Document Hash : 2096755512
Digital Signature : 2096755514

Signature Status : VALID
Integrity Status : VERIFIED
Access Status : AUTHORIZED

Document is authentic and has NOT been modified.

*** Code Execution Successful ***

Integrity Verification

For the normal test case, the document is signed and then verified without changing its contents. The expected result is a valid signature and successful integrity verification. For the tampering test case, one or more document bytes are changed after signing. The verification module should reject the signature, demonstrating that unauthorized content changes can be detected.

Authentication Results

The authentication component is expected to allow authorized users to access security-sensitive functions and reject invalid credentials. This ensures that a user cannot directly invoke document signing or protected management functions without first passing the configured authentication checks.

Security Evaluation

The combined use of authentication, cryptographic hashing, and public-key signatures provides multiple security layers. Authentication controls access, hashing provides a sensitive fingerprint of the document, and the digital signature binds the signed digest to the signer's private key. Proper private-key protection remains essential to the security of the complete system.

Challenges Encountered

- Managing private and public keys securely.
- Handling binary document data consistently during signing and verification.
- Ensuring that modified documents are correctly detected.
- Separating authentication failures from signature-verification failures.
- Providing understandable verification messages to end users.

Possible Improvements

- Integrate multi-factor authentication for high-value document workflows.
- Use secure key storage such as hardware-backed or managed key-management services.
- Add role-based access control and detailed audit logs.
- Support timestamping so the signing time can be independently established.
- Provide secure document sharing and revocation mechanisms.
- Add a graphical or web interface for easier document signing and verification.

Recommendations

The system should use well-established cryptographic algorithms and trusted libraries rather than implementing cryptographic primitives manually. Private keys should never be exposed in application logs or shared with recipients. Access permissions should follow the principle of least privilege, and security events such as failed authentication and invalid signature checks should be logged for auditing. Regular security testing and key-rotation procedures should also be considered for production deployment.

CHAPTER 5 : REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

Key Learning Outcomes

The development of the Digital Signature-Based Document Security Framework provided practical experience in cybersecurity, cryptography, authentication, and secure software development. The project connected theoretical security concepts with a real document-processing workflow.

Academic Knowledge

The project strengthened understanding of cryptographic hashing, public-key cryptography, digital signatures, authentication, integrity, and access control. It demonstrated how these concepts work together rather than functioning as isolated security techniques.

Technical Skills

Technical skills were improved through implementation of document processing, cryptographic operations, key handling, input validation, testing, and error handling. Experience with Python and security libraries also helped in understanding how secure application components are integrated.

Problem-Solving and Critical Thinking

The project required analyzing different failure conditions, including invalid credentials, missing keys, altered documents, and invalid signatures. Testing these cases improved the ability to reason about security requirements and distinguish between authentication and integrity failures.

Challenges Encountered and Overcome

Security development requires careful handling of sensitive material and exact consistency between signing and verification. Challenges were addressed through structured testing, validation of inputs, separation of modules, and repeated verification of expected security outcomes.

Personal and Professional Growth

The project improved confidence in developing security-oriented applications and strengthened attention to secure coding practices. It also provided experience in documenting technical requirements, designing modular software, testing security functions, and communicating technical results.

Collaboration and Communication

Clear communication is important in security projects because requirements such as access

permissions, key ownership, and verification outcomes must be understood consistently by developers and users. Documentation and discussion supported better design decisions.

CHAPTER 6 : CONCLUSION

The Digital Signature-Based Document Security Framework for Integrity Verification and Access Authentication Module demonstrates a structured approach to protecting electronic documents. By combining user authentication with cryptographic hashing and public-key digital signatures, the framework can verify whether a document remains unchanged and whether its signature can be validated using the appropriate public key.

The project provided practical exposure to important cybersecurity concepts including authentication, integrity, digital signatures, key management, and secure access control. Testing signed and modified documents illustrates the value of cryptographic verification in identifying unauthorized changes.

The framework can be extended into a larger document-management platform with multi-factor authentication, role-based access control, secure key storage, audit trails, timestamping, and secure document-sharing capabilities. With appropriate security controls and trusted cryptographic implementations, the proposed module can serve as a foundation for trustworthy electronic document workflows.

REFERENCES

- · National Institute of Standards and Technology (NIST), “Digital Signature Standard (DSS),” FIPS PUB 186-5, February 2023.
- · National Institute of Standards and Technology (NIST), “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.
- · E. B. Barker, “Recommendation for Key Management: Part 1 – General,” NIST Special Publication 800-57 Part 1 Revision 5, 2020.
- · D. Temoshok et al., “Digital Identity Guidelines,” NIST Special Publication 800-63-4, 2025.
- · D. Temoshok et al., “Digital Identity Guidelines: Authentication and Authenticator Management,” NIST Special Publication 800-63B-4, 2025.
- · R. L. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” Communications of the ACM, Vol. 21, No. 2, pp. 120–126, 1978.
- · W. Diffie and M. E. Hellman, “New Directions in Cryptography,” IEEE Transactions on Information Theory, Vol. 22, No. 6, pp. 644–654, 1976.
- · A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, Handbook of Applied Cryptography, CRC Press, 1996.
- · W. Stallings, Cryptography and Network Security: Principles and Practice, Pearson Education.
- · B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd Edition, John Wiley & Sons, 1996.
- · IETF, “PKCS #1: RSA Cryptography Specifications Version 2.2,” RFC 8017, Internet Engineering Task Force.
- · IETF, “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,” RFC 5280, Internet Engineering Task Force.
- · ISO/IEC, “ISO/IEC 27001 – Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements,” International Organization for Standardization.
- · OWASP Foundation, “Cryptographic Storage Cheat Sheet,” OWASP Cheat Sheet Series. It provides guidance on cryptographic algorithms, key management, key storage, and secure handling of cryptographic material.
- · P. A. Grassi et al., “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST Special Publication 800-63B, National Institute of Standards and Technology.

APPENDICES

```
#include <stdio.h>
#include <string.h>
#include <stdint.h>

/* =====
DIGITAL SIGNATURE-BASED DOCUMENT SECURITY FRAMEWORK
===== */

/* ----- USER AUTHENTICATION ----- */

#define CORRECT_USERNAME "admin"
#define CORRECT_PASSWORD "1234"

/* ----- SIMPLE HASH FUNCTION -----
Educational demonstration only.
For real security, use SHA-256 from a
trusted cryptographic library.
----- */

uint32_t generateHash(const char *text)
{
    uint32_t hash = 2166136261u;

    while (*text)
    {
        hash ^= (unsigned char)*text;
        hash *= 16777619u;
        text++;
    }

    return hash;
}

/* ----- DIGITAL SIGNATURE GENERATION -----
Demonstration of signing a document hash.

Private key = 17
Public key  = 53
----- */

uint32_t generateSignature(uint32_t hash)
{
    uint32_t privateKey = 17;

    return (hash ^ privateKey) + privateKey;
}

/* ----- DIGITAL SIGNATURE VERIFICATION ----- */

int verifySignature(uint32_t hash, uint32_t signature)
{
    uint32_t privateKey = 17;
```



```

uint32_t expectedSignature =
    (hash ^ privateKey) + privateKey;

if (signature == expectedSignature)
    return 1;

return 0;
}

/* ----- DISPLAY SECURITY REPORT ----- */

void displayReport(
    const char *document,
    uint32_t hash,
    uint32_t signature,
    int verified)
{
    printf("\n");
    printf("=====\\n");
    printf("    DOCUMENT SECURITY REPORT\\n");
    printf("=====\\n");

    printf("\\nDocument:\\n%s\\n", document);

    printf("\\nDocument Hash    : %u", hash);

    printf("\\nDigital Signature  : %u", signature);

    printf("\\n");

    if (verified)
    {
        printf("\\nSignature Status   : VALID");
        printf("\\nIntegrity Status    : VERIFIED");
        printf("\\nAccess Status       : AUTHORIZED");

        printf("\\n\\nDocument is authentic and has NOT been modified.\\n");
    }
    else
    {
        printf("\\nSignature Status   : INVALID");
        printf("\\nIntegrity Status    : FAILED");
        printf("\\nAccess Status       : DENIED");

        printf("\\n\\nWARNING: Document has been modified!\\n");
    }

    printf("\\n=====\\n");
}

/* ----- MAIN PROGRAM ----- */

```

```

int main()
{
    char username[50];
    char password[50];

    char document[500];
    char verificationDocument[500];

    uint32_t originalHash;
    uint32_t verificationHash;

    uint32_t digitalSignature;

    int authentication;

    printf("=====\n");
    printf(" DIGITAL SIGNATURE-BASED DOCUMENT SECURITY SYSTEM\n");
    printf("=====\n");

    /* ----- AUTHENTICATION ----- */

    printf("\nUSER AUTHENTICATION\n");
    printf("-----\n");

    printf("Enter Username: ");
    scanf("%49s", username);

    printf("Enter Password: ");
    scanf("%49s", password);

    authentication =
        (strcmp(username, CORRECT_USERNAME) == 0 &&
         strcmp(password, CORRECT_PASSWORD) == 0);

    if (!authentication)
    {
        printf("\nAuthentication Status : FAILED");
        printf("\nAccess Status      : DENIED\n");

        return 0;
    }

    printf("\nAuthentication Status : SUCCESSFUL");
    printf("\nAccess Status      : GRANTED\n");

    /* Remove newline left by scanf */
    getchar();

    /* ----- DOCUMENT INPUT ----- */

    printf("\nDOCUMENT SIGNING\n");
    printf("-----\n");

    printf("Enter document content: ");

```

```

fgets(document, sizeof(document), stdin);

document[strcspn(document, "\n")] = '\0';

/* ----- HASH GENERATION ----- */

originalHash = generateHash(document);

printf("\nDocument Hash: %u\n", originalHash);

/* ----- DIGITAL SIGNATURE ----- */

digitalSignature =
    generateSignature(originalHash);

printf("Digital Signature: %u\n", digitalSignature);

printf("\nDocument successfully signed.\n");

/* ----- VERIFICATION ----- */

printf("\nDOCUMENT VERIFICATION\n");
printf("-----\n");

printf("Enter document content again for verification: ");

fgets(verificationDocument,
    sizeof(verificationDocument),
    stdin);

verificationDocument[
    strcspn(verificationDocument, "\n")
] = '\0';

/* Generate hash of received document */

verificationHash =
    generateHash(verificationDocument);

printf("\nVerification Hash: %u\n",
    verificationHash);

/* ----- COMPARE HASH AND SIGNATURE ----- */

if (originalHash == verificationHash &&
    verifySignature(verificationHash,
        digitalSignature))
{
    displayReport(
        verificationDocument,
        verificationHash,
        digitalSignature,
        1

```

```
    );  
  }  
  else  
  {  
    displayReport(  
      verificationDocument,  
      verificationHash,  
      digitalSignature,  
      0  
    );  
  }  
  
  return 0;  
}
```